Adult Census Income End-to-End 데이터 분석 · 종합 실습 2

실행 결과 정리 및 프로젝트 회고

소속	판교 캠퍼스 9반 3조
팀원	김명철, 김민정, 김진수, 정민성, 전진만 (가나다순)
데이터셋	UCI Adult Census Income (32,561행 × 15열)
실행 스크립트	src/main.py

저희 팀은 UCI Adult Census Income 데이터셋으로 데이터 준비부터 시각화, 통계 분석, ML Pipeline, 보고서 자동화까지 이어지는 End-to-End 분석 파이프라인을 구현했습니다. 이 문서는 `src/main.py`를 실행하면 순서대로 나오는 화면을 정리하고, 프로젝트를 진행하며 느낀 점을 담았습니다.

1. 실행 결과 정리

00 실행 시작 · 환경 기록

실행을 시작하면 팀 정보와 로그 파일 경로를 먼저 출력하고, 이어서 Python 3.12.0 / pandas 3.0.5 / polars 1.43.2 / scikit-learn 1.9.0 등 실행 환경의 라이브러리 버전을 기록합니다. random_state=42로 고정되어 있어 같은 환경이면 동일한 수치가 재현됩니다.

```
$ 실행 시작

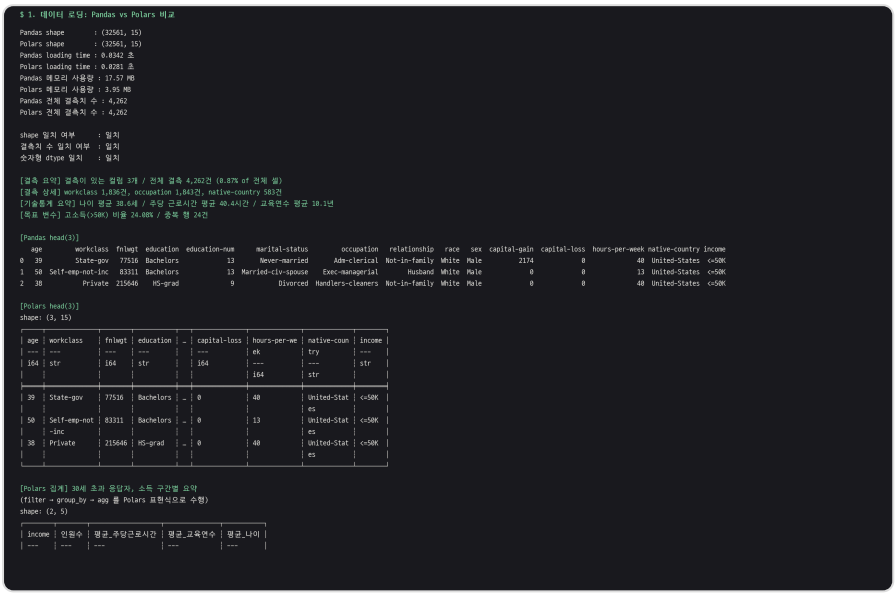
[종합 실습 2] Adult Census Income 분석을 시작합니다.
판교 9반 3조 — 김명철, 김민정, 김진수, 정민성, 전진만
로그 파일: /Users/violet/WebstormProjects/판교_9반_3조_day2종합실습/outputs/logs/run.log
```

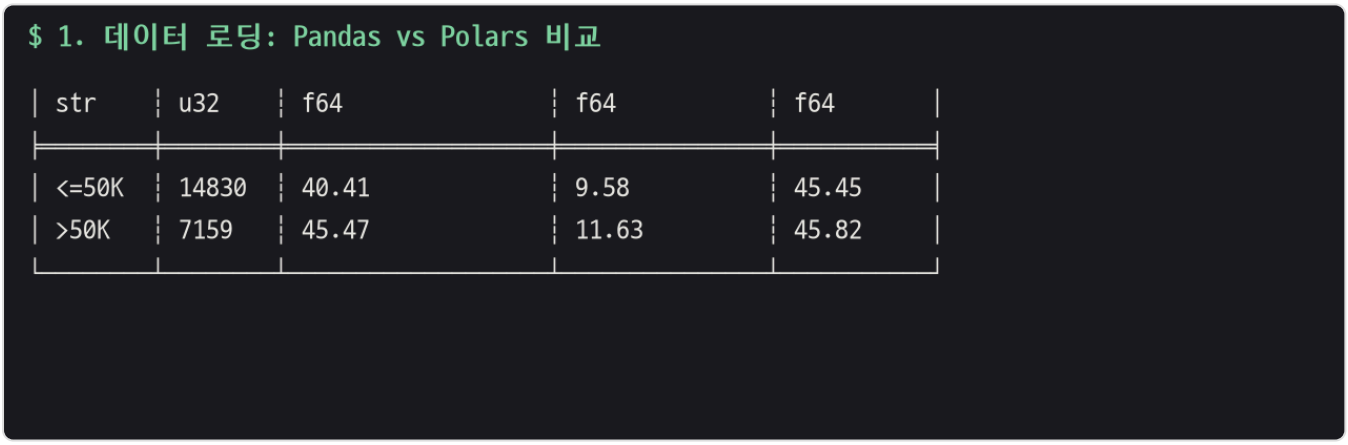
```
$ 0. 실행 환경

Python        : 3.12.0
OS            : Darwin 24.1.0
pandas        : 3.0.5
polars        : 1.43.2
numpy         : 2.5.1
scikit-learn  : 1.9.0
scipy         : 1.18.0
matplotlib    : 3.11.1
seaborn       : 0.13.2
plotly        : 6.9.0
joblib        : 1.5.3
실행 파일 경로 : /Users/violet/WebstormProjects/판교_9반_3조_day2종합실습/src/logging_setup.py
작업 기준 경로 : /Users/violet/WebstormProjects/판교_9반_3조_day2종합실습
한글 폰트 적용: AppleGothic
[다운로드] https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data
[다운로드 완료] /Users/violet/WebstormProjects/판교_9반_3조_day2종합실습/data/raw/adult.data (3,974,305 bytes)
```

01 데이터 로딩 — Pandas vs Polars 비교

동일한 로컬 파일을 두 라이브러리로 각각 읽어 shape·로딩 시간·메모리·결측치 수를 비교합니다. shape (32,561 × 15), 결측치 4,262건이 두 라이브러리에서 모두 일치했고, 메모리 사용량은 Polars가 3.95MB로 Pandas(17.57MB)보다 적었습니다. 단순 비교에 그치지 않고 Polars의 filter → group_by → agg 표현식으로 30세 초과 응답자의 소득 구간별 집계까지 별도로 수행했습니다.





02 결측치 · 중복 데이터 확인 및 처리

결측치는 workclass(1,836건) · occupation(1,843건) · native-country(583건), 총 4,262건이 확인되었습니다. 중복 행 24건은 제거했고, 범주형 결측은 EDA에서는 원본 분포 보존을 위해 그대로 두고 ML 단계의 SimpleImputer로 대치하는 방식을 택했습니다. 처리 후 32,537행이 남았습니다.

```
$ 2. 결측치 / 중복 데이터 확인 및 처리

[컬럼별 결측치 수]
age                  0
workclass         1836
fnlwgt               0
education            0
education-num        0
marital-status       0
occupation        1843
relationship         0
race                 0
sex                  0
capital-gain         0
capital-loss         0
hours-per-week       0
native-country     583
income               0

결측치가 있는 컬럼 수 : 3
전체 결측치 수        : 4,262
중복 행 개수          : 24

원본 행 수                    : 32,561
중복 제거 후 행 수            : 32,537 (제거 24행)
목표 변수 결측 제거 후 행 수  : 32,537 (제거 0행)
처리 후 남은 결측치 수        : 4,261 (범주형 3개 컬럼, ML Pipeline에서 최빈값으로 대치)
income 클래스 목록            : ['<=50K', '>50K']
```

03 기본 EDA

income 분포는 <=50K 75.91%(24,698건), >50K 24.09%(7,839건)로 약 3.15:1의 클래스 불균형이 확인되었습니다. education·occupation·sex 등 주요 범주형 변수의 빈도도 함께 출력했습니다.

```
$ 3. 기본 EDA

[데이터 구조]
행 수 : 32,537 / 열 수 : 15

[dtypes]
age              int64
workclass          str
fnlwgt           int64
education          str
education-num    int64
marital-status     str
occupation         str
relationship       str
race               str
sex                str
capital-gain     int64
capital-loss     int64
hours-per-week   int64
native-country     str
income             str

[income 분포]
 <=50K :  24,698건 (75.91%)
  >50K :   7,839건 (24.09%)
클래스 불균형 비율(다수/소수) : 3.15 : 1

[education 빈도 상위 5개]
education
HS-grad         10494
Some-college     7282
Bachelors        5353
Masters          1722
Assoc-voc        1382

[occupation 빈도 상위 5개, 결측 제외]
occupation
Prof-specialty     4136
Craft-repair       4094
Exec-managerial    4065
Adm-clerical       3768
Sales              3650

[sex 빈도]
sex
Male      21775
Female    10762
```

04 기술통계 — 평균 · 표준편차 · 분위수

age, education-num, capital-gain, capital-loss, hours-per-week 5개 수치형 변수에 대해 describe() 기반 평균·표준편차·사분위수를 산출했습니다. 예: age 평균 38.59세(표준편차 13.64), hours-per-week 평균 40.44시간.

```
$ 4. 기술통계: 평균 / 표준편차 / 분위수

                  count     mean      std   min   25%   50%   75%       max
age            32,537.00    38.59    13.64 17.00 28.00 37.00 48.00     90.00
education-num  32,537.00    10.08     2.57  1.00  9.00 10.00 12.00     16.00
capital-gain   32,537.00 1,078.44 7,387.96  0.00  0.00  0.00  0.00 99,999.00
capital-loss   32,537.00    87.37   403.10  0.00  0.00  0.00  0.00  4,356.00
hours-per-week 32,537.00    40.44    12.35  1.00 40.00 40.00 45.00     99.00

[소득 집단별 주요 지표 평균]
         age  education-num  hours-per-week
income
<=50K  36.79           9.60           38.84
>50K   44.25          11.61           45.47
```

05 상관분석 — 수치형 변수 상관계수

수치형 변수 간 피어슨 상관계수를 계산했습니다. income_binary와 상관이 가장 큰 변수는 education-num(0.335), 그다음이 age(0.234)였습니다. 히트맵으로도 시각화했습니다.

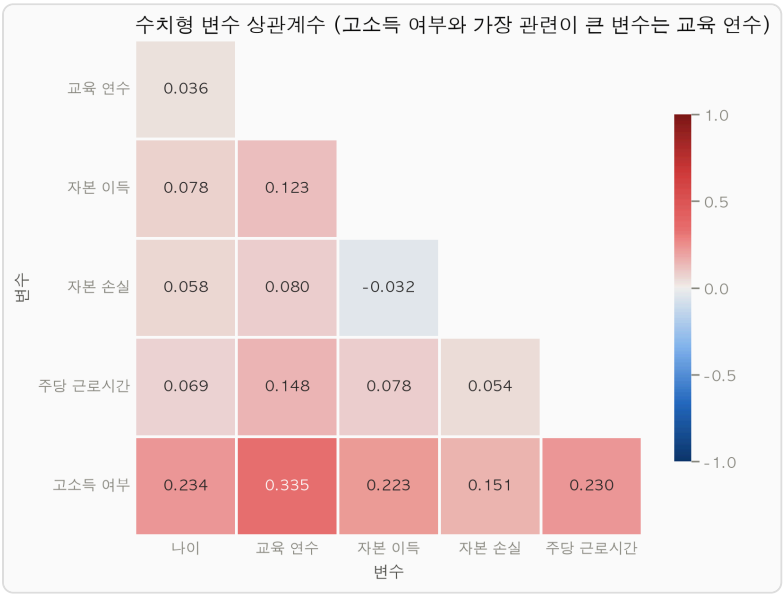
```
$ 5. 상관분석: 수치형 변수 상관계수

                   age  education-num  capital-gain  capital-loss  hours-per-week  income_binary
age             1.0000         0.0362        0.0777        0.0577          0.0685         0.2340
education-num   0.0362         1.0000        0.1227        0.0799          0.1484         0.3353
capital-gain    0.0777         0.1227        1.0000       -0.0316          0.0784         0.2233
capital-loss    0.0577         0.0799       -0.0316        1.0000          0.0542         0.1505
hours-per-week  0.0685         0.1484        0.0784        0.0542          1.0000         0.2297
income_binary   0.2340         0.3353        0.2233        0.1505          0.2297         1.0000

[고소득 여부(income_binary)와의 상관계수, 절댓값 큰 순]
education-num     0.3353
age               0.2340
hours-per-week    0.2297
capital-gain      0.2233
capital-loss      0.1505

피어슨 상관계수는 선형 관계만 잡아낸다. 수치형 변수만 대상이므로,
범주형 변수와 목표 변수의 관계는 다음 단계에서 카이제곱 검정으로 따로 확인한다.

[저장 완료] 03_correlation_heatmap.png
```



06 범주형 변수 연관성 — 카이제곱 검정 · Cramér's V

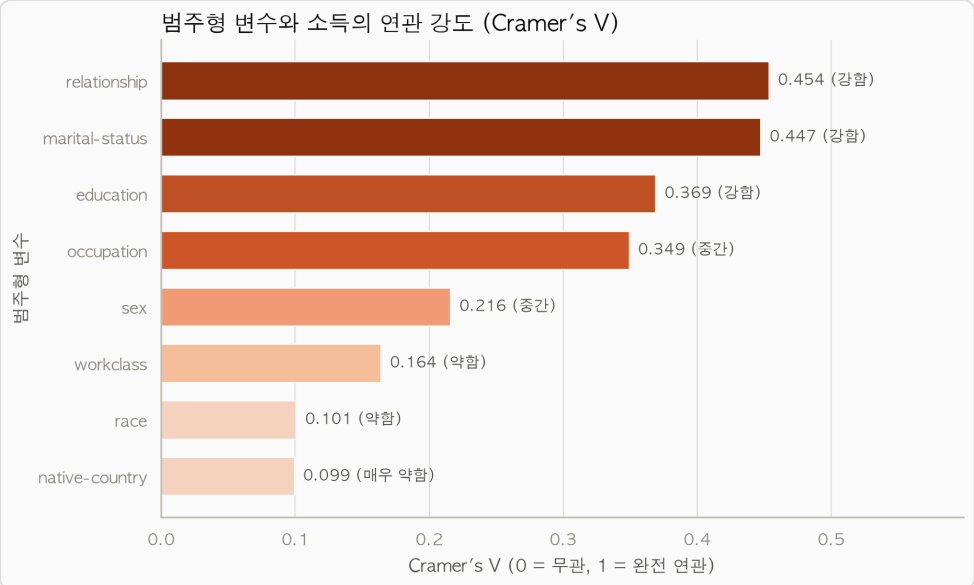
8개 범주형 변수 각각에 대해 income과의 카이제곱 독립성 검정을 수행하고, 연관 강도는 Cramér's V로 별도 측정했습니다. relationship(V=0.4535, 강함)이 가장 강하게 연관되었고, native-country(V=0.0993, 매우 약함)가 가장 약했습니다. 표본이 3만 건대라 p-value는 대부분 유의수준 이하로 나오므로, 유의성 여부보다 Cramér's V 크기로 해석하도록 안내하고 있습니다.

```
$ 6. 범주형 변수 연관성 분석 (카이제곱 검정 · Cramér's V)

H0 : 해당 범주형 변수와 income은 서로 독립이다(관계가 없다).
H1 : 두 변수는 독립이 아니다(관계가 있다).
유의수준 : 0.05

[범주형 변수와 income의 연관성, Cramér's V 큰 순]
           변수  범주 수    카이제곱  자유도                          p-value Cramers_V 연관 강도 유의성
  relationship     6 6,692.1    5 1e-308 미만 (부동소수점 표현 한계로 0.0으로 반환됨)    0.4535    강함 유의함
 marital-status     7 6,510.3    6 1e-308 미만 (부동소수점 표현 한계로 0.0으로 반환됨)    0.4473    강함 유의함
     education    16 4,428.4   15 1e-308 미만 (부동소수점 표현 한계로 0.0으로 반환됨)    0.3689    강함 유의함
    occupation    14 3,742.8   13 1e-308 미만 (부동소수점 표현 한계로 0.0으로 반환됨)    0.3492    중간 유의함
           sex     2 1,516.5    1 1e-308 미만 (부동소수점 표현 한계로 0.0으로 반환됨)    0.2159    중간 유의함
     workclass     8   826.4    7                     3.65366e-174    0.1641    약함 유의함
          race     5   330.9    4                      2.27979e-70    0.1009    약함 유의함
native-country    41   315.3   40                        1.827e-44    0.0993 매우 약함 유의함

가장 강하게 연관된 변수는 relationship 입니다 (Cramér's V = 0.4535, 강함).
표본이 3만 건대라 대부분의 변수에서 p-value가 매우 작게 나온다.
따라서 '유의한가'보다 Cramér's V로 '얼마나 강한가'를 보는 것이 실질적이다.
[저장 완료] 04_categorical_association.png
```



07 t-test — 소득 집단별 주당 근로시간 평균 비교

정규성(Shapiro-Wilk 부분표본 + 왜도·첨도)과 등분산성(Levene) 검정을 먼저 수행한 뒤, 등분산이 기각되어 Welch's t-test(equal_var=False)를 적용했습니다. 결과: t=45.095, p<1e-308(표현 한계 미만), 평균 차이 6.63시간/주, Cohen's d=0.552(중간 효과). 표본이 매우 커서 p-value만으로는 차이의 크기를 알 수 없다는 점을 짚고 효과크기를 함께 제시한 점이 특징입니다.

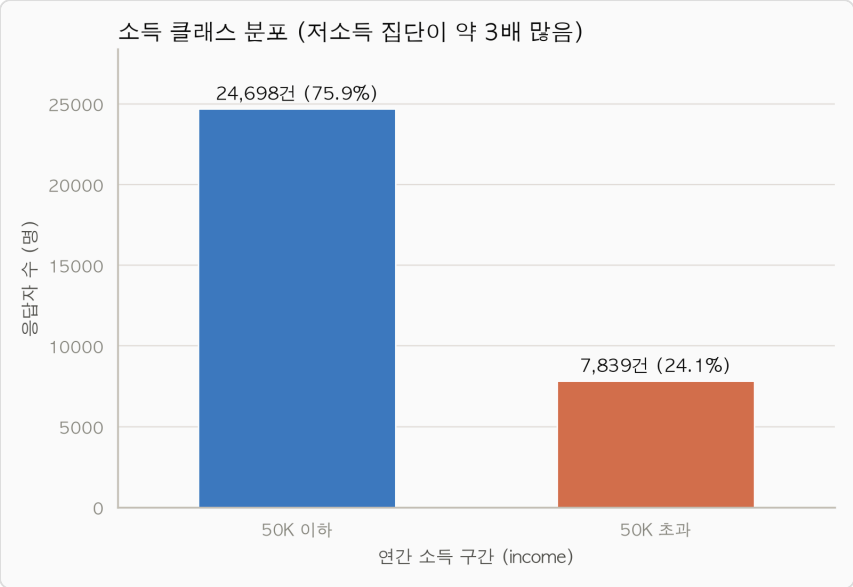


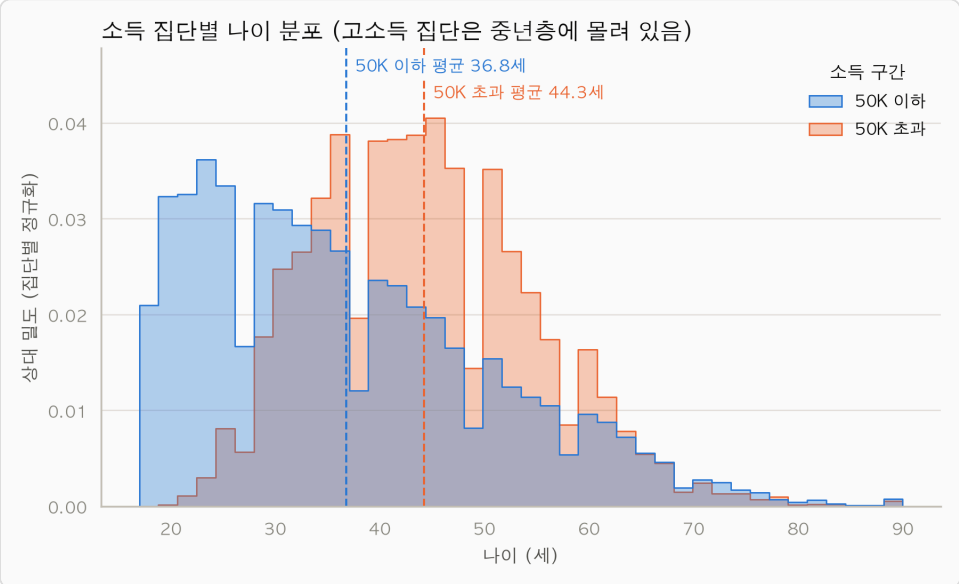
08 Seaborn 정적 차트 생성

소득 클래스 분포, 소득 집단별 나이 분포, 상관계수 히트맵, 범주형 연관 강도까지 총 4종의 Seaborn 정적 차트를 생성했습니다. 모든 차트에 제목과 x·y축 레이블이 한글로 포함되어 있습니다.

```
$ 8. Seaborn 정적 차트 생성

[저장 완료] 01_income_distribution.png
[저장 완료] 02_age_by_income.png
```



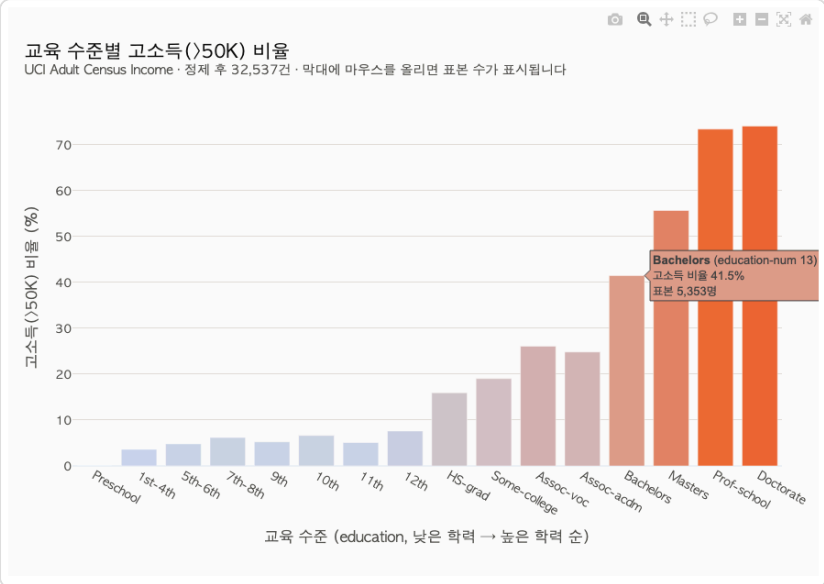


09 Plotly 인터랙티브 차트 생성

교육 수준별 고소득(>50K) 비율을 Plotly 막대 차트로 생성해 outputs/figures/05_income_by_education.html로 저장했습니다. hover 시 교육 연수·표본 수까지 툴팁으로 표시됩니다. 아래는 실제로 HTML을 열어 Bachelors 막대에 마우스를 올린 화면입니다.

```
$ 9. Plotly 인터랙티브 차트 생성

[저장 완료] 05_income_by_education.html
브라우저에서 열어 hover tooltip이 보이는 화면을 캡처하세요.
```



10 Machine Learning Pipeline — 학습 · 평가 · 저장

ColumnTransformer(수치형: SimpleImputer→StandardScaler / 범주형: SimpleImputer→OneHotEncoder) + LogisticRegression 구조의 Pipeline을 학습했습니다. 기본 모델 Accuracy 0.8563 / Precision 0.7418 / Recall 0.6193 / F1 0.6750 / ROC-AUC 0.9097이며, class_weight='balanced' 모델과 나란히 비교하고, 5-겹 교차검증(F1 0.6583±0.0052)으로 결과의 안정성도 확인했습니다. 두 Pipeline을 joblib으로 저장한 뒤 재로딩하여 예측값이 저장 전과 동일한지까지 검증합니다.


```
$ 10. Machine Learning Pipeline: 학습 · 평가 · 저장

학습 데이터 수 : 26,029 / 평가 데이터 수 : 6,508
학습 세트 클래스 비율 : <=50K 75.91% / >50K 24.09%

[기본 모델] LogisticRegression (class_weight 없음)
Accuracy  : 0.8563
F1 Score  : 0.6750
Precision : 0.7418
Recall    : 0.6193
ROC-AUC   : 0.9097

[Classification Report]
              precision    recall  f1-score   support

       <=50K     0.8852    0.9316    0.9078      4940
        >50K     0.7418    0.6193    0.6750      1568

    accuracy                         0.8563      6508
   macro avg     0.8135    0.7754    0.7914      6508
weighted avg     0.8506    0.8563    0.8517      6508

[Confusion Matrix]  행=실제, 열=예측
            예측 <=50K  예측 >50K
실제 <=50K      4602       338
실제 >50K        597       971

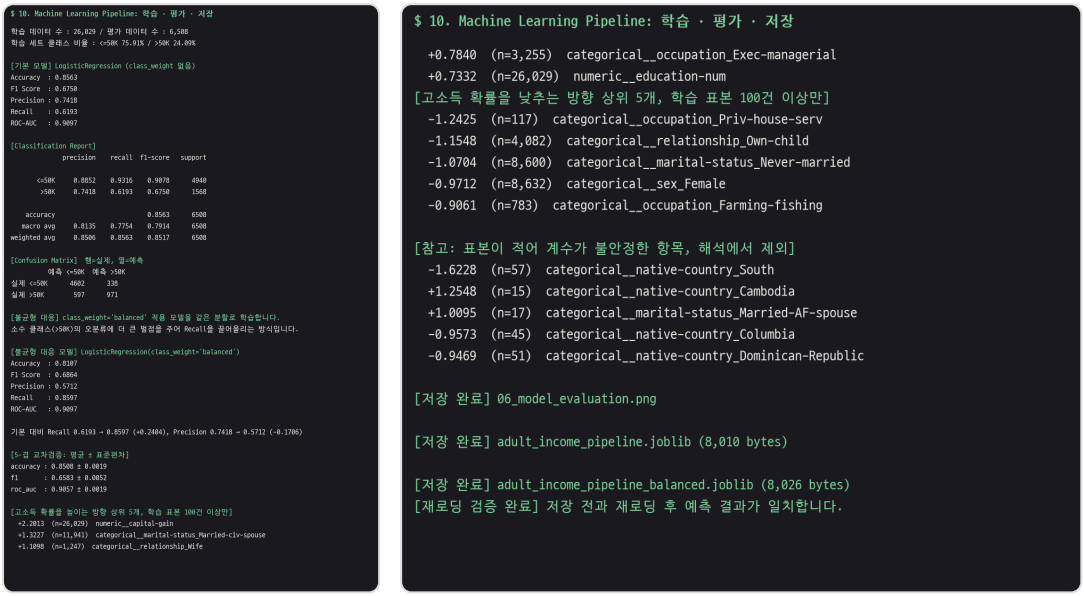
[불균형 대응] class_weight='balanced' 적용 모델을 같은 분할로 학습합니다.
소수 클래스(>50K)의 오분류에 더 큰 벌점을 주어 Recall을 끌어올리는 방식입니다.

[불균형 대응 모델] LogisticRegression(class_weight='balanced')
Accuracy  : 0.8187
F1 Score  : 0.6864
Precision : 0.5712
Recall    : 0.8597
ROC-AUC   : 0.9097

기본 대비 Recall 0.6193 → 0.8597 (+0.2404), Precision 0.7418 → 0.5712 (-0.1706)

[5-겹 교차검증: 평균 ± 표준편차]
accuracy : 0.8508 ± 0.0019
f1       : 0.6583 ± 0.0052
roc_auc  : 0.9057 ± 0.0019

[고소득 확률을 높이는 방향 상위 5개, 학습 표본 100건 이상만]
  +2.2013  (n=26,029)  numeric__capital-gain
  +1.3227  (n=11,941)  categorical__marital-status_Married-civ-spouse
  +1.1098  (n=1,247)  categorical__relationship_Wife
```



```
$ 10. Machine Learning Pipeline: 학습 · 평가 · 저장

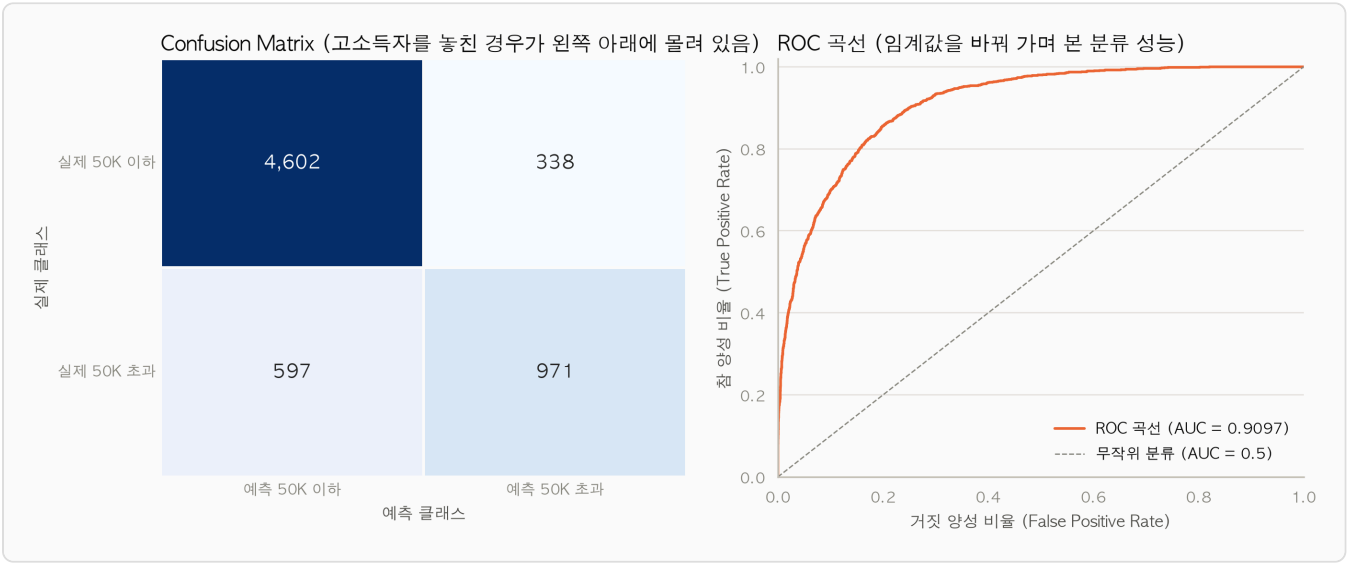
  +0.7840  (n=3,255)  categorical__occupation_Exec-managerial
  +0.7332  (n=26,029)  numeric__education-num
[고소득 확률을 낮추는 방향 상위 5개, 학습 표본 100건 이상만]
  -1.2425  (n=117)  categorical__occupation_Priv-house-serv
  -1.1548  (n=4,082)  categorical__relationship_Own-child
  -1.0704  (n=8,600)  categorical__marital-status_Never-married
  -0.9712  (n=8,632)  categorical__sex_Female
  -0.9061  (n=783)  categorical__occupation_Farming-fishing

[참고: 표본이 적어 계수가 불안정한 항목, 해석에서 제외]
  -1.6228  (n=57)  categorical__native-country_South
  +1.2548  (n=15)  categorical__native-country_Cambodia
  +1.0095  (n=17)  categorical__marital-status_Married-AF-spouse
  -0.9573  (n=45)  categorical__native-country_Columbia
  -0.9469  (n=51)  categorical__native-country_Dominican-Republic

[저장 완료] 06_model_evaluation.png

[저장 완료] adult_income_pipeline.joblib (8,010 bytes)

[저장 완료] adult_income_pipeline_balanced.joblib (8,026 bytes)
[재로딩 검증 완료] 저장 전과 재로딩 후 예측 결과가 일치합니다.
```



11 심화 분석 — 파생변수 · 모델 비교 · 임계값 조정

연령대 구간, 자본이득/손실 발생 여부 등 파생변수를 추가하고 결측을 'Unknown' 범주로 남긴 모델, class_weight 적용 모델, 트리 기반 HistGradientBoosting 모델까지 4가지를 동일 분할에서 비교했습니다. F1 기준 최고는 HistGradientBoosting(0.7246), Recall 기준 최고는 class_weight=balanced 모델(0.8667)이었습니다. 판정 임계값을 0.05~0.95로 바꿔가며 Precision·Recall·F1의 변화도 분석했고, F1이 최대가 되는 임계값은 0.32였습니다.

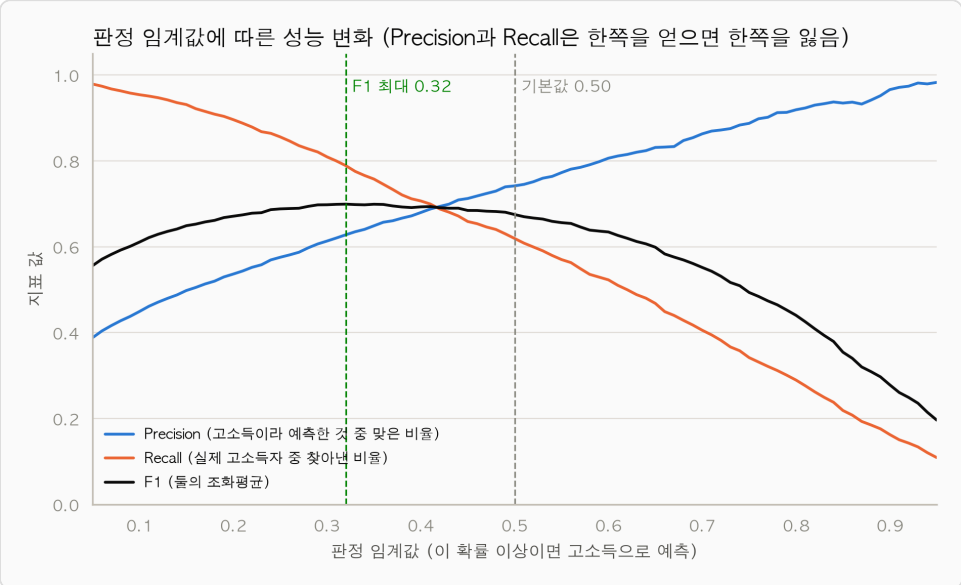
```
$ 11. 심화 분석: 파생변수, 모델 비교, 임계값 조정

[학습 완료] ① 기본 (로지스틱 회귀)
[학습 완료] ② + 파생변수 · Unknown 범주
[학습 완료] ③ + class_weight=balanced
[학습 완료] ④ 트리 기반 (HistGradientBoosting)

[모델 비교: 동일한 학습·평가 분할 기준]
                          accuracy  precision  recall     f1  roc_auc
① 기본 (로지스틱 회귀)                  0.8563     0.7418  0.6193 0.6750   0.9097
② + 파생변수 · Unknown 범주             0.8596     0.7477  0.6295 0.6835   0.9169
③ + class_weight=balanced         0.8173     0.5810  0.8667 0.6957   0.9172
④ 트리 기반 (HistGradientBoosting)    0.8772     0.7884  0.6703 0.7246   0.9317

F1 최고     : ④ 트리 기반 (HistGradientBoosting) (0.7246)
Recall 최고: ③ + class_weight=balanced (0.8667)

[임계값 조정: 기본 모델]
기본값 0.50    : Recall 0.6193
F1 최대 임계값 : 0.32 → F1 0.6993 / Precision 0.6279 / Recall 0.7889
Recall 0.80 달성: 임계값 0.30 → Precision 0.6133 (정밀도를 그만큼 포기해야 함)
[저장 완료] 07_threshold_tradeoff.png
```



12 report.md 자동 생성

위 모든 단계의 실제 실행 결과값만을 사용해 outputs/report.md(약 24KB)를 자동 생성합니다. 보고서용으로 수치를 별도 계산하지 않고 각 단계가 반환한 값을 그대로 문자열에 삽입하는 방식이라, 화면 출력과 문서 수치가 항상 일치합니다.

```
$ 12. outputs/report.md 자동 생성

[생성 완료] /Users/violet/WebstormProjects/판교_9반_3조_day2종합실습/outputs/report.md (24,517 bytes)
```

13 산출물 생성 확인 · 캡쳐본 생성

차트 7종, 모델 2종, report.md, 로그 파일 등 기대되는 산출물 10종이 모두 정상 생성(파일 크기 0바이트 아님)되었는지 검증합니다. 이어서 실행 로그를 파싱해 보고서용 캡쳐본 24개 파일을 자동 생성합니다.

```
$ 산출물 생성 확인

생성 완료: outputs/figures/01_income_distribution.png (69,884 bytes)
생성 완료: outputs/figures/02_age_by_income.png (94,169 bytes)
생성 완료: outputs/figures/03_correlation_heatmap.png (104,278 bytes)
생성 완료: outputs/figures/04_categorical_association.png (114,870 bytes)
생성 완료: outputs/figures/05_income_by_education.html (4,861,182 bytes)
생성 완료: outputs/figures/06_model_evaluation.png (182,968 bytes)
생성 완료: outputs/figures/07_threshold_tradeoff.png (186,462 bytes)
생성 완료: outputs/model/adult_income_pipeline.joblib (8,010 bytes)
생성 완료: outputs/model/adult_income_pipeline_balanced.joblib (8,026 bytes)
생성 완료: outputs/report.md (24,517 bytes)
```

```
$ 13. 보고서용 캡쳐본 생성

캡쳐본 폰트: D2Coding
```

2. 프로젝트를 진행하며 느낀 점

2-1. 데이터 분석을 처음 공부하며 느낀 점

사실 저는 이번 프로젝트를 하기 전까지 데이터 분석을 제대로 공부해본 적이 없었습니다. 평균·표준편차·상관계수·가설검정 같은 용어는 어디선가 들어본 정도였지, 그게 실제로 코드 몇 줄로 어떻게 계산되고 왜 그런 순서로 써야 하는지는 전혀 모르는 상태였습니다. 그래서 이번에 `pandas` · `polars`로 데이터를 직접 읽고, `describe()`로 평균·표준편차·분위수를 뽑아보고, `scipy.stats`로 가설검정을 돌려보고, `sklearn.pipeline.Pipeline`으로 전처리와 모델을 하나로 묶어보는 과정을 처음부터 끝까지 손으로 직접 짜보면서, 구체적으로 아래와 같은 것들을 새로 알게 되었습니다.

- **같은 데이터라도 라이브러리마다 읽는 방식이 다르다는 것.** Pandas는 `na_values="?"`, `skipinitialspace=True` 옵션만 주면 끝나는데, Polars에는 그 옵션이 없어서 문자열 앞뒤 공백을 직접 `strip_chars()`로 제거하고 `"?"`를 null로 바꾸고 숫자 컬럼을 다시 캐스팅하는 과정을 손으로 넣어야 Pandas와 같은 결과가 나온다는 걸 직접 코드를 맞춰보고 나서야 알았습니다.
- **`describe()`가 찍어주는 숫자를 그냥 넘기면 안 된다는 것.** capital-gain의 평균이 1,078인데 중앙값(50%)은 0으로 나오는 걸 보고, 대부분이 0원이고 일부 극단값이 평균을 끌어올린 희소 분포라는 걸 알게 되었고, 그래서 나중에 `has-capital-gain`처럼 "발생 여부"를 나타내는 파생 변수를 따로 만들어야 한다는 것까지 이어서 이해하게 됐습니다.
- **Pipeline으로 묶지 않으면 학습 때와 예측 때 전처리가 어긋날 수 있다는 것.** 전처리(`SimpleImputer`, `StandardScaler`, `OneHotEncoder`)와 모델을 따로 코드에 흩어 두면, 나중에 새 데이터를 예측할 때 같은 순서로 전

처리했는지 사람이 일일이 확인해야 합니다. `Pipeline` 객체 하나로 묶고 `joblib`으로 저장한 뒤 다시 불러와 예측값을 비교해보고 나서야, 왜 이 구조가 필요한지 실감이 났습니다.

특히 좋았던 건, 이론으로만 알던 개념이 실제 데이터를 만나는 순간 훨씬 더 와닿았다는 점입니다. "상관계수가 높다"는 문장을 책으로 읽을 때와, income과 education-num의 상관계수가 0.335로 나온 걸 직접 보고 "그럼 이게 income과 가장 관련 있는 수치형 변수구나"라고 스스로 해석해보는 건 완전히 다른 경험이었습니다. Pandas와 Polars를 나란히 돌려서 로딩 시간과 메모리 사용량 차이를 직접 눈으로 확인한 것도, 그냥 "Polars가 더 빠르다더라"는 얘기만 듣는 것과는 이해의 깊이가 달랐던 것 같습니다.

2-2. 제가 맡은 부분 — 통계 분석(기술통계·상관계수·t-test)을 구현하며

저는 이번 과제 요구사항 중 세 번째인 **통계 분석** 부분, 즉 기술통계·상관계수·t-test와 p-value 해석을 맡아서 구현했습니다. 처음에는 "scipy.stats.ttest_ind 한 줄만 부르면 끝나는 거 아닌가" 하고 쉽게 생각했는데, 막상 자료를 찾아보고 구현해보니 생각보다 훨씬 신경 쓸 게 많다는 걸 알게 되었습니다.

- **t-test를 쓰려면 먼저 확인해야 할 게 있다는 걸 처음 알았습니다.** t-test는 정규성과 등분산성이라는 전제 조건이 있다는 걸 이번에 처음 제대로 배웠습니다. 그래서 Shapiro-Wilk로 정규성을, Levene으로 등분산성을 먼저 검정하고 그 결과에 따라 Student's t-test와 Welch's t-test 중 어느 것을 쓸지 정하도록 구현했습니다. 근로시간 데이터가 40시간에 몰려 있는 비대칭 분포라는 걸 직접 확인하고 나니, 왜 정규성 가정이 중요한지 몸으로 이해가 됐습니다.
- **표본이 크면 검정 방법 자체가 흔들린다는 것도 새로 배운 부분이었습니다.** Shapiro-Wilk를 표본 32,537건에 그대로 돌렸더니 아주 작은 차이에도 계속 정규성이 기각되는 걸 보고 처음엔 당황했습니다. 찾아보니 Shapiro-Wilk는 표본이 커지면 민감해지는 성질이 있다는 걸 알게 되었고, 그래서 부분표본으로 검정하면서 왜도·첨도를 같이 보고, 중심극한정리에 따르면 표본이 충분히 크면 표본평균의 분포가 정규에 가까워진다는 논리로 근거를 세워서 t-test를 진행하도록 만들었습니다. 이렇게 실제 데이터에서 막히는 지점을 만나고 그걸 해결해나가는 과정이 제일 기억에 남습니다.
- **"통계적으로 유의하다"와 "차이가 크다"는 다른 말이라는 걸 이번에 체감했습니다.** 표본이 3만 건대다 보니 p-value가 사실상 거의 항상 0에 가깝게 나온다는 걸 확인하고, p-value만으로는 아무 의미가 없다는 걸 깨달았습니다. 그래서 Cohen's d로 효과크기를 따로 계산해서 함께 제시하도록 구현했는데, 이 과정에서 "유의확률"과 "효과의 크기"가 왜 따로 필요한 개념인지 스스로 정리가 됐습니다.
- **범주형 변수는 상관계수를 못 쓴다는 것도 직접 코드를 짜다가 알게 됐습니다.** 처음에는 모든 변수에 `corr()`를 돌리면 되는 줄 알았는데, 범주형 변수(workclass, occupation 등)에는 피어슨 상관계수를 쓸 수 없다는 걸 알고 나서 카이제곱 검정과 Cramér's V로 따로 구현했습니다. 카이제곱 검정이 "관계가 있다/없다"만 알려주고 "얼마나 강한가"는 알려주지 못한다는 것도 이번에 처음 알게 된 부분이라, 그 둘을 같이 써야 하는 이유를 직접 코드로 짜보면서 이해했습니다.
- **p-value가 0.0으로 찍히는 걸 보고 부동소수점의 한계도 처음 알게 됐습니다.** t-test 결과 p-value가 그냥 0.0으로 나와서 처음엔 코드가 잘못된 줄 알았는데, 찾아보니 double 자료형이 표현할 수 있는 가장 작은 값(약 1e-308)보다 작아서 생기는 현상이라는 걸 알게 되었습니다. 이런 사소해 보이는 문제를 만나고 원인을 찾아가는 과정도 이번 프로젝트에서 얻은 소득이라고 생각합니다.

2-3. 팀원들과 GitHub으로 협업하며 쌓은 경험

이번 프로젝트는 데이터 준비, 시각화, 통계 분석, ML Pipeline, 자동화·발표까지 다섯 파트를 팀원 다섯 명이 나눠서 맡고, 각자 만든 코드를 GitHub으로 합치는 방식으로 진행했습니다. 혼자 코드를 짤 때는 몰랐던 것들을 이 과정에서 많이 배웠습니다.

가장 크게 부딪힌 건 `merge conflict`였습니다. 특히 `main.py`는 전체 실행 흐름을 조립하는 파일이라 다섯 명이 다 같이 손을 댔고, 그만큼 충돌도 잦았습니다. 충돌이 날 때마다 각자의 변경 사항을 확인해서 합치는 걸 반복하면서 git으로 충돌을 해결하는 경험을 쌓았습니다.

또 팀원들의 코드를 직접 읽어볼 기회가 많았던 것도 좋았습니다. 제가 맡은 통계 분석 부분과 맞닿아 있는 시각화 코드나, 제가 만든 결과를 그대로 받아서 쓰는 ML Pipeline·보고서 자동화 코드를 보면서, 제가 만든 함수가 어떤 형태의 값을 반환해야 다른 팀원이 쓰기 편한지를 고민하게 됐습니다.

2-4. 마무리하며

데이터 분석을 처음 배우는 입장에서, 이번 프로젝트는 통계와 머신러닝 개념을 직접 손으로 구현해보면서 익힐 수 있었던 경험이었습니다. 특히 제가 맡았던 통계 분석 부분을 통해 가설검정의 전제 조건, 효과크기, 표본 크기의 함정 같은 것들을 몸으로 배울 수 있었고, 팀원들과 GitHub으로 협업하면서는 코드를 잘 짜는 것만큼이나 다른 사람과 맞춰가는 과정도 중요하다는 걸 느꼈습니다. 이번 경험을 바탕으로 다음 프로젝트에서는 더 나은 방식으로 협업하고, 더 깊이 있게 통계와 모델링을 공부해보고 싶습니다.

3. 팀별 종합의견

이번 프로젝트에서는 Adult Census Income 데이터셋을 활용하여 데이터 로딩부터 전처리, 탐색적 데이터 분석, 시각화, 통계 분석, 머신러닝 모델 구축 및 결과 보고서 자동 생성까지 데이터 분석의 전체 과정을 End-to-End 형태로 구현하였다. Pandas와 Polars를 활용하여 동일한 데이터를 처리하고 성능과 메모리 사용량을 비교함으로써 데이터 규모와 활용 목적에 따라 적절한 라이브러리를 선택하는 것이 중요하다는 점을 확인하였다. 또한 결측치와 중복 데이터를 처리하고, 기술통계와 상관분석, t-test 등의 통계적 분석을 수행하면서 단순히 수치를 산출하는 것뿐만 아니라 분석 결과를 적절하게 해석하는 과정이 중요하다는 점을 이해할 수 있었다. 특히 표본의 크기가 큰 경우 p-value만으로 결과를 판단하기보다 효과크기와 변수 간 연관성을 함께 고려해야 한다는 점을 확인하였다.

머신러닝 단계에서는 전처리와 모델 학습을 Pipeline으로 구성하여 동일한 처리 과정이 반복적으로 적용될 수 있도록 하였으며, 클래스 불균형 문제를 고려하여 Accuracy뿐만 아니라 Precision, Recall, F1-Score, ROC-AUC 등 다양한 평가 지표를 함께 비교하였다. 또한 class weight와 Decision Threshold를 조정하면서 하나의 평가 지표만을 기준으로 모델을 판단하기보다는 실제 활용 목적에 따라 적절한 기준을 선택해야 함을 알 수 있었다. 분석 결과를 report.md로 자동 생성하고 모델을 저장·재로딩하는 과정까지 구현함으로써 분석의 재현성과 자동화의 중요성도 확인할 수 있었다. 이번 프로젝트를 통해 데이터 분석은 개별 기능을 구현하는 것에서 끝나는 것이 아니라 데이터 준비, 통계적 해석, 모델링, 평가 및 결과 관리가 하나의 흐름으로 연결되어야 한다는 점을 배울 수 있었으며, 향후에는 다양한 모델 비교와 변수 선택, 다중공선성 검토 및 실제 비즈니스 목적을 반영한 평가 기준 등을 추가하여 분석의 완성도를 더욱 높일 수 있을 것으로 생각한다.